Содержание

1 Описание предметной области 3
1.1 Общие положения 3
1.2 Сведения из теории 3
2 Технология разработки приложения 5
2.1 Алгоритм решения 5
2.2 Описание программы 9
2.3 Результат работы программы 17
3 Инструкция по пользованию проектом 20
Заключение 28
Список использованных источников 29

1 Описание предметной области

1.1 Общие положения

Сетевое приложение предназначено для использования сотрудниками и покупателями оптового контура по закупке продукции отдела сбыта фермерского хозяйства, включая следующие роли пользователей:

- Директор магазина (покупатель) — добавляет свои адреса для доставки и заказывает на эти адреса продукцию фермерского хозяйства с видением статуса обработки.
- Менеджер — добавляет, редактирует и удаляет товары для заказов, подтверждает заявки и сопровождает учётные записи директоров.
- Логист — координирует подтверждённые заказы и распределяет их между водителями также обновляя их статус, и контролирует выполнение доставок.
- Водитель — выполняет доставку заказов, отмечает выполнение заказа обновляя его статус.

Целью курсового проекта является проектирование и реализация сетевого приложения для автоматизации работы отдела сбыта фермерского хозяйства с публичным доступом через сайт https://farmer.indevs.in.

1.2 Сведения из теории

Проект представляет собой сетевое веб-приложение с отдельной клиентской частью и backend-модулем, который собирается как исполняемый Spring Boot JAR. Выбор конкретных инструментов продиктован необходимостью обеспечения целостности данных, удобства поддержки кода и автоматизации процессов сборки.

Java 21 — объектно-ориентированный язык программирования общего назначения.

- *Обоснование выбора:* Java выбрана в качестве основного языка разработки благодаря строгой типизации и мощной системе управления памятью, что минимизирует количество ошибок времени выполнения. Кроссплатформенность Java позволяет развёртывать приложение на различных серверных окружениях без изменения кода.

IntelliJ IDEA — интегрированная среда разработки (IDE) от JetBrains.

- *Обоснование выбора:* Использование IntelliJ IDEA обусловлено наличием продвинутых инструментов статического анализа кода, рефакторинга и отладки. Это ускоряет разработку сложных модулей и помогает поддерживать высокое качество исходного кода.

Spring Boot — фреймворк для создания серверной части веб-приложения и REST API.

- *Обоснование выбора:* Выбор в пользу Spring Boot сделан для обеспечения переносимой серверной архитектуры, удобной обработки HTTP-запросов, маршрутизации, валидации и конфигурации безопасности.

React — компонентная библиотека для построения пользовательского интерфейса.

- *Обоснование выбора:* React выбран для реализации уровня представления с целью соблюдения принципа разделения ответственности. Компонентный подход позволяет избежать дублирования разметки и сделать интерфейсы ролей более поддерживаемыми.

Maven — инструмент управления проектами и сборки.

- *Обоснование выбора:* Maven используется для автоматизации процесса сборки backend-модуля и управления зависимостями. Это гарантирует воспроизводимость сборки на любом компьютере разработчика и упрощает подключение необходимых библиотек без конфликтов версий.

PostgreSQL — объектно-реляционная СУБД с открытым исходным кодом.

- *Обоснование выбора:* Для хранения данных проекта выбрана PostgreSQL, поскольку она поддерживает свойства ACID, надёжную работу с конкурентными операциями и удобные выборки по каталогу, пользователям, заказам и маршрутам доставки.

1.3 Постановка задачи

Целью данной курсовой работы является разработка сетевого приложения для автоматизации работы отдела сбыта фермерского хозяйства, позволяющего автоматизировать основные бизнес-процессы, такие как управление заказами, подтверждение заявок, распределение маршрутов и завершение доставки. Приложение должно обеспечивать эффективное взаимодействие между пользователями с различными ролями — директором магазина, менеджером, логистом и водителем.

Основные задачи разработки:

- Создать интерфейс для каждой роли пользователя, включающий функционал, необходимый для выполнения их задач: директор магазина — создание и контроль заказов; менеджер — настройка каталога товаров, создание учётных записей директоров и одобрение заказов; логист — распределение заказов по водителям; водитель — перевозка заказа и подтверждение доставки.

- Реализовать механизм координации доставки: организацию распределения заявок между водителями, статусную модель заказа и предварительный просмотр результата транспортной задачи.

- Обеспечить безопасность и доступность приложения: внедрить систему аутентификации и разграничения прав доступа для пользователей с разными ролями, а также подготовить публичную развёрнутую версию сайта для проверки на защите.

Использовать Java и JavaScript в разработке серверной и клиентской частей приложения, а также среду разработки IntelliJ IDEA для создания, тестирования и отладки кода. Для визуальной составляющей использовать React, CSS и JavaScript.

Решение поставленных задач позволит создать приложение, которое оптимизирует процесс оформления и доставки заказов, улучшит координацию между сотрудниками фермерского хозяйства и повысит общую эффективность работы отдела сбыта.

Требования к оборудованию и программному обеспечению для разработчика, использующего IntelliJ IDEA 2025.3, включают:

Процессор не ниже AMD Ryzen 3 или аналогичный. Оперативная память минимум 8 ГБ, SSD-накопитель для быстрой работы с проектом и экран с разрешением не ниже 1920×1080 обеспечат удобную разработку и показ публичной версии системы.

Для разработчика необходимо наличие интернет-соединения для загрузки зависимостей, обновлений программного обеспечения и документации. Это соединение также позволяет использовать системы контроля версий и получать актуальные компоненты проекта.

Операционная система должна быть Microsoft Windows 10 64-bit и выше. Для проверки публичной версии сайта достаточно современного браузера и доступа к сети Интернет.

Для сопровождения и повторного развёртывания приложения требуется поддержка JDK 21, Node.js 20, доступ к репозиторию и настроенным переменным окружения backend и frontend-сервисов.

Эти требования обеспечат стабильную и эффективную работу разработчика при создании, тестировании и защите сетевого приложения.

Для пользователя требования к оборудованию включают наличие компьютера или мобильного устройства с доступом к сети и браузером, поддерживающим современные веб-технологии.

Требования к браузеру включают использование последних версий Google Chrome, Microsoft Edge, Mozilla Firefox или других совместимых решений. Это обеспечивает корректное отображение интерфейса, карт, таблиц и пользовательских форм.

Для корректной работы сценариев доставки и транспортной задачи также рекомендуется стабильное интернет-соединение, позволяющее загружать картографические данные, обновлять статусы заказов и выполнять выгрузку отчётности без задержек.

2 Технология разработки приложения

2.1 Алгоритм решения

Алгоритм проектирования сетевого приложения:

1 Создать в базе данных таблицы PostgreSQL:

1.1 Создать в базе данных таблицу users;

1.2 Создать в базе данных таблицу store_addresses;

1.3 Создать в базе данных таблицу products;

1.4 Создать в базе данных таблицу orders;

1.5 Создать в базе данных таблицу order_items;

1.6 Создать таблицы stock_movements, order_timeline_events и realtime_notifications.

2 Реализовать серверную часть на Java и Spring Boot:

2.1 Создать проект Spring Boot и настроить Maven, profiles и pom.xml;

2.2 Реализовать сущности model: User, StoreAddress, Product, Order, OrderItem, OrderTimelineEvent, RealtimeNotification;

2.3 Реализовать слой repository и миграции схемы базы данных;

2.4 Реализовать security, JWT-аутентификацию и правила доступа;

2.5 Реализовать AuthController и production-настройки JWT-аутентификации;

2.6 Реализовать GeoController, NotificationController и DashboardController;

2.7 Реализовать OrderController для ручного и автоматического назначения маршрутов;

2.8 Реализовать ProductController, UserManagementController и ReportController;

2.9 Реализовать DirectorProfileController, сервисы профиля, заказов и маршрутизации.

2.10 Подключить frontend API и интеграцию React-клиента с backend-контуром.

3 Подготовить пользовательский интерфейс, публичное развёртывание и материалы для проверки;

После этого выполняется развёртывание backend-сервиса, frontend-сайта и проверка основных пользовательских потоков на публичном адресе.

Финальным этапом выступает проверка публичного домена, учётных записей и эксплуатационных инструкций для работы с задеплоенным сайтом.

Для начала необходимо подготовить публичную версию проекта. Frontend разворачивается как static site и доступен по адресу https://farmer.indevs.in, а backend работает отдельным сервисом https://farm-sales-backend.onrender.com/api. Связь между ними задаётся переменными окружения и правилами CORS (рисунок 2.1).

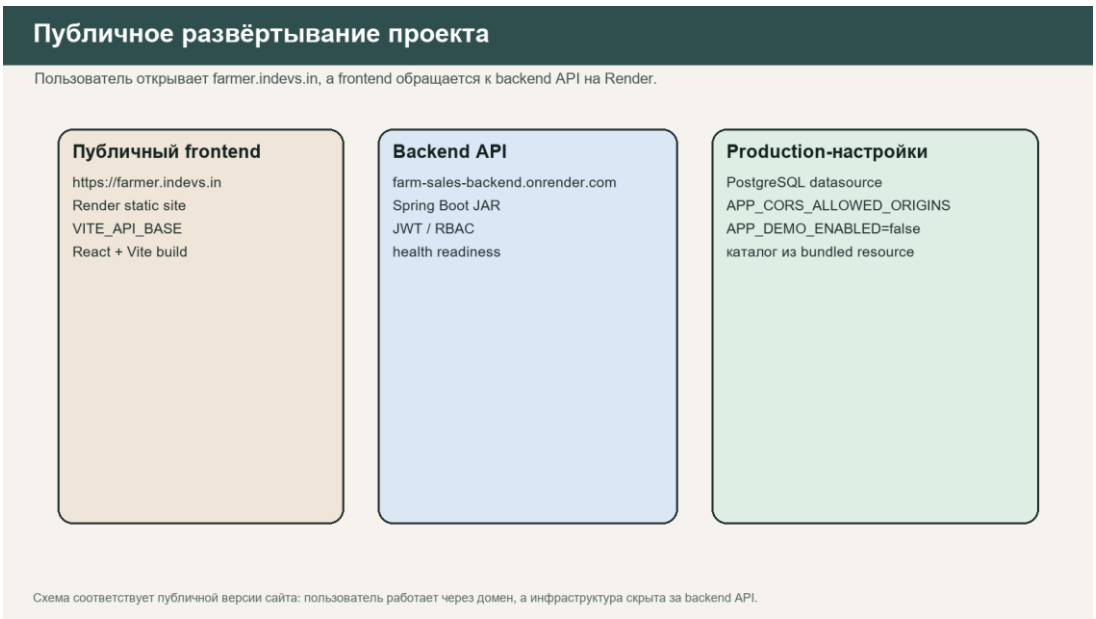

Рисунок 2.1 – Схема публичного развёртывания сайта

База данных Farm Sales сетевого приложения состоит из таблиц users, store_addresses, products, orders, order_items, stock_movements, order_timeline_events и realtime_notifications. Каждая таблица выполняет собственную функцию, а вместе они образуют полный контур сопровождения заказа и логистики (рисунок 2.2).

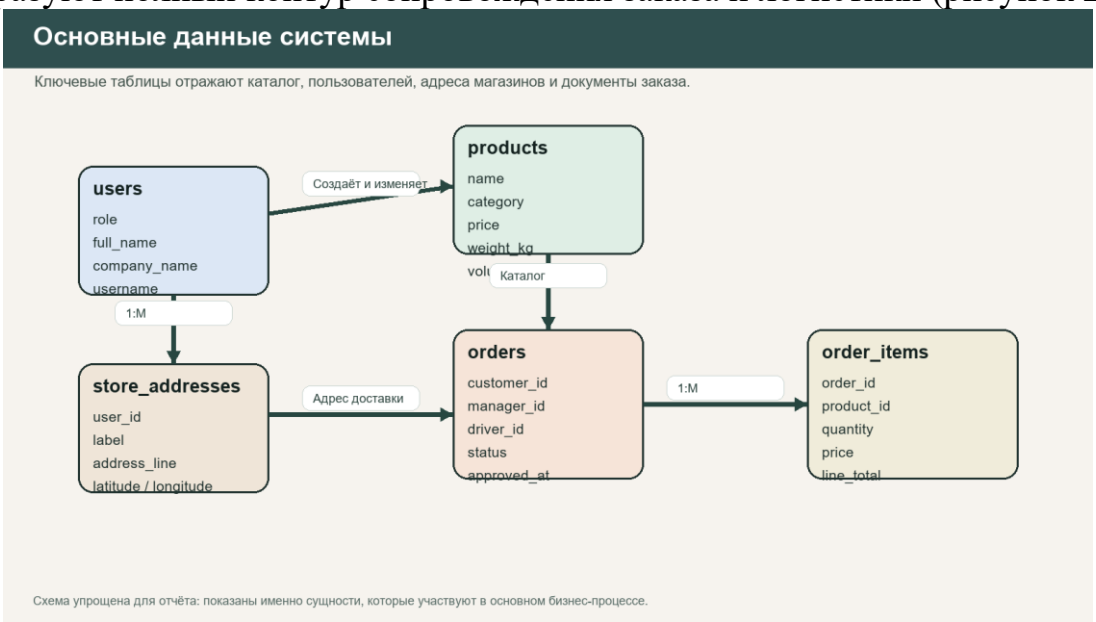

Рисунок 2.2 – База данных Farm Sales

Таблица users – хранит пользователей системы и их роли

id – идентификатор пользователя, первичный ключ

username – уникальный логин для входа в систему

full_name – имя сотрудника или представителя магазина

role – роль пользователя (DIRECTOR, MANAGER, LOGISTICIAN, DRIVER)

company_name – наименование магазина или организации

phone – контактный телефон

Таблица store_addresses – хранит адреса доставки магазинов

id – идентификатор адреса, первичный ключ

user_id – ссылка на владельца адреса

label – краткое название точки доставки

address_line – человекочитаемый адрес

latitude / longitude – координаты для карты и маршрута

Таблица products – хранит каталог фермерской продукции

id – идентификатор товара, первичный ключ

name – название продукции

category – категория товара

price – цена единицы продукции

weight_kg и volume_m3 – логистические параметры для расчёта маршрута

photo_url – ссылка на карточку изображения

Таблица orders – хранит документы заказа

id – идентификатор заказа, первичный ключ

customer_id – ссылка на директора магазина

manager_id – менеджер, обработавший заявку

driver_id – назначенный водитель

status – состояние заказа (CREATED, APPROVED, ASSIGNED, DELIVERED)

created_at и approved_at – временные метки жизненного цикла

Таблица order_items – хранит позиции внутри заказа

order_id – ссылка на документ заказа

product_id – ссылка на товар

quantity – количество товара в заказе

price – цена товара на момент оформления

Таблицы stock_movements, order_timeline_events и realtime_notifications – обслуживают историю изменений, движение остатков и уведомления интерфейса

2.2 Описание программы

Приложение имеет модульную структуру, где корневой уровень содержит deploy-конфигурацию, документацию и эксплуатационные файлы, а логика системы разделена между backend и frontend. Такая организация соответствует реальной структуре репозитория, представленной на рисунке 2.3.

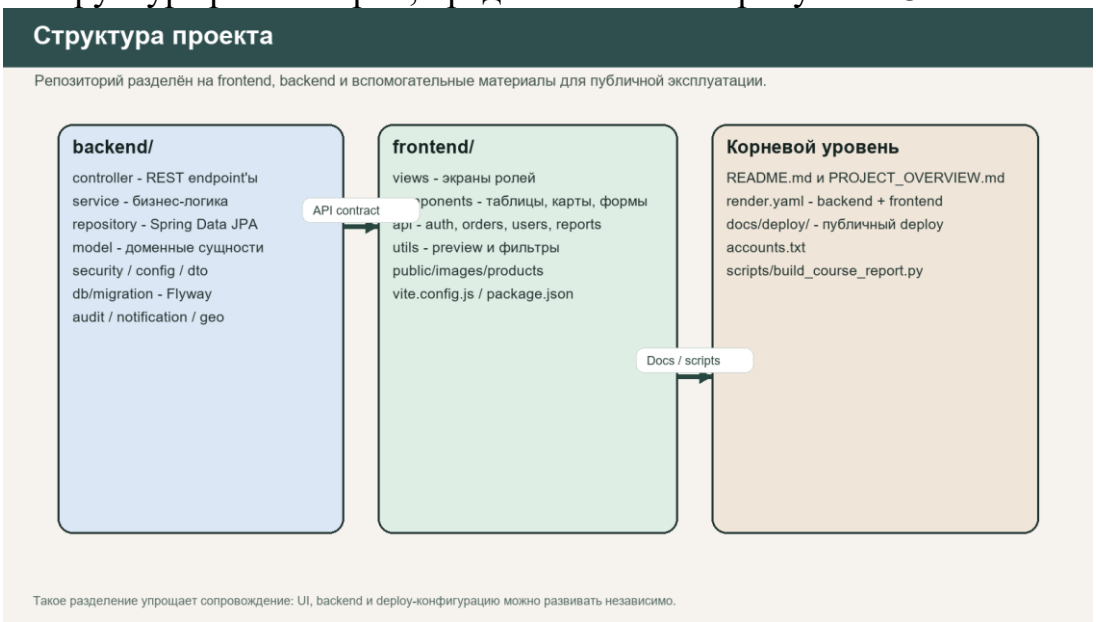

Рисунок 2.3 – Структура приложения

Папка backend/src/main/java/com/farm/sales содержит основную программную логику приложения.

Именно в ней реализованы серверная часть веб-приложения, работа с базой данных, обработка запросов пользователей, бизнес-логика системы, транспортная задача, отчёты и публикация уведомлений. Структура данной папки организована по пакетам, каждый из которых отвечает за отдельную подсистему приложения.

Пакет model содержит классы-сущности предметной области, которые используются для хранения данных, получаемых из базы данных, а также для передачи этих данных между слоями приложения.

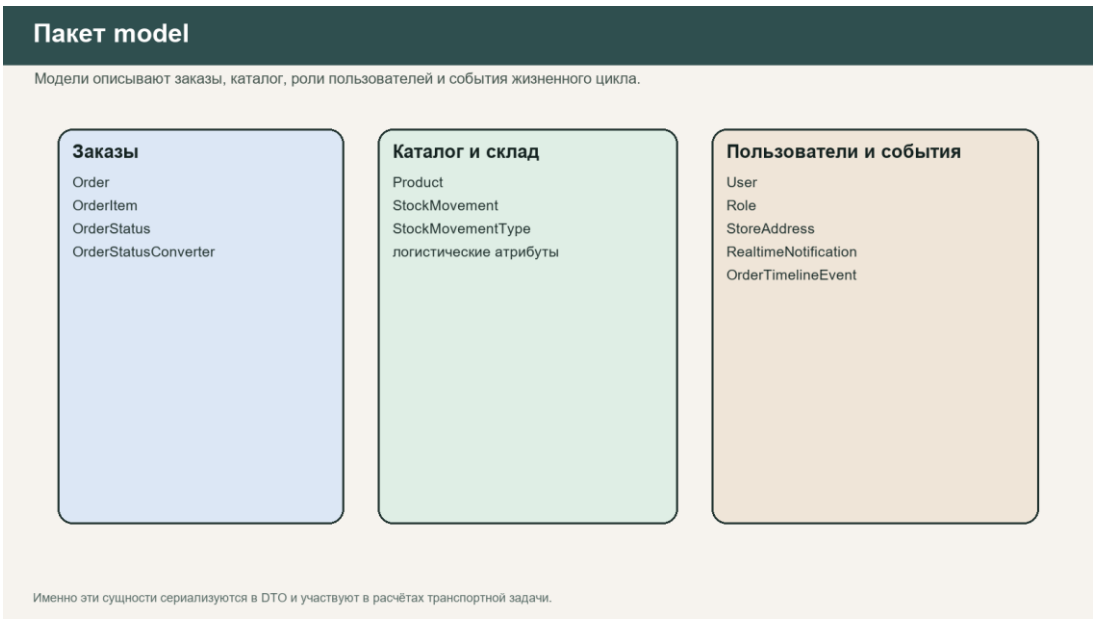

Рисунок 2.4 – Пакет model

В данном пакете расположены следующие сущности:

User и Role — модель пользователя системы. Хранят сведения об учётной записи: идентификатор, логин, пароль, роль, имя пользователя и контактные данные, необходимые для авторизации и персонализации интерфейса.

StoreAddress — модель адреса доставки. Содержит данные об адресе клиента, а также географические координаты, которые используются в модуле маршрутизации.

Product — модель товара. Содержит информацию о товаре: название, описание, цену, изображение, категорию, а также логистические параметры, например вес и объём, которые далее участвуют в расчёте доставки.

Order — модель заказа. Содержит информацию о заказе, его статусе, привязке к клиенту, водителю, адресу, времени создания и другим служебным данным.

OrderItem — модель позиции заказа. Представляет отдельный товар внутри заказа, его количество, цену и связь с конкретным заказом.

OrderTimelineEvent — модель записи истории, фиксирующая подтверждение, назначение маршрута и завершение доставки.

RealtimeNotification — модель уведомления, используемая для передачи служебных событий между backend и пользовательским интерфейсом.

Связи между указанными сущностями отражают реальные процессы отдела сбыта: один директор магазина может иметь несколько адресов доставки, один заказ включает множество позиций товара, а каждое изменение статуса сопровождается записью в истории. Благодаря такому описанию предметной области бизнес-операции backend остаются прозрачными и предсказуемыми.

Таким образом, пакет model формирует объектное представление данных системы и является основой для работы остальных модулей приложения.

Пакет repository реализует слой доступа к данным, то есть взаимодействие приложения с PostgreSQL. В нём сосредоточены классы, выполняющие выборку, сохранение и агрегацию данных для пользовательских сценариев.

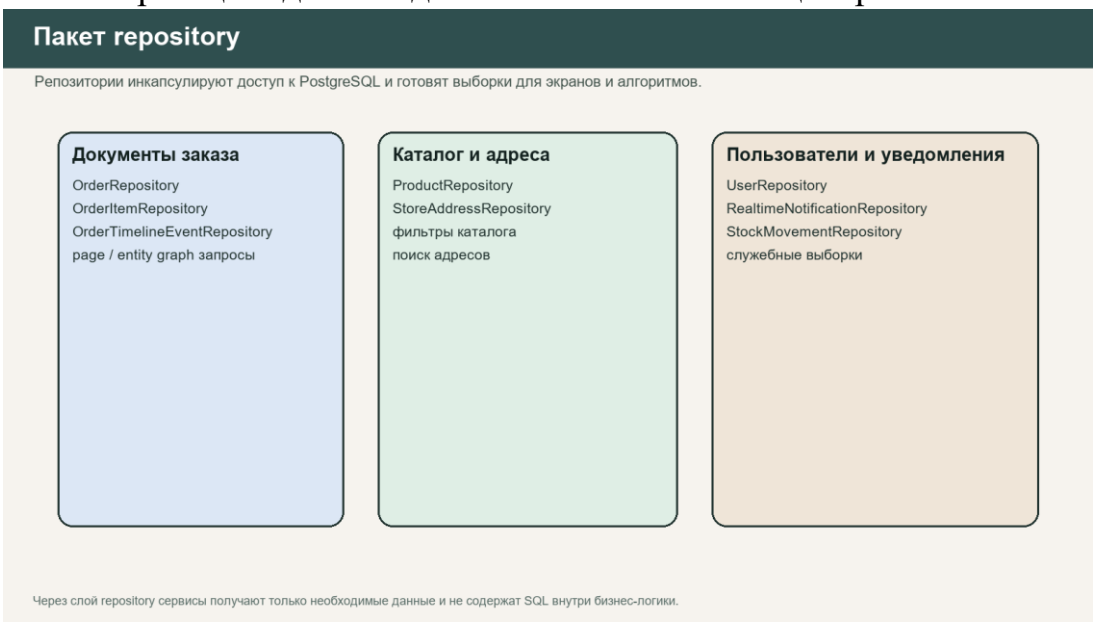

Рисунок 2.5 – Пакет repository

В пакет входят следующие классы:

OrderRepository — отвечает за работу с документами заказа, их поиском, фильтрацией по статусам и выдачей данных для логистики.

OrderItemRepository — сопровождает позиции заказа и обеспечивает выборку состава документа по идентификатору заказа.

ProductRepository — используется для фильтрации каталога, поиска товаров и выдачи ассортимента в клиентские экраны.

StoreAddressRepository — выполняет выборку адресов магазинов и точек доставки, связанных с директором магазина.

UserRepository — используется при регистрации, поиске и сопровождении пользователей различных ролей.

OrderTimelineEventRepository — хранит историю изменений документа и обеспечивает последующее восстановление жизненного цикла заказа.

RealtimeNotificationRepository — выдаёт служебные уведомления для обновления интерфейса без ручной перезагрузки страниц.

StockMovementRepository — используется для служебных выборок по остаткам и аналитике движения товарных позиций.

Часть репозиториев использует стандартные механизмы Spring Data JPA, а часть содержит специализированные выборки для аналитики и распределения заказов. Это особенно важно для логистического модуля, где необходимо быстро получить только подтверждённые заявки, данные по водителям и агрегированные показатели маршрутов.

Слой repository выступает связующим звеном между объектной моделью приложения и реальными данными, хранящимися в базе.

Пакет controller содержит REST-контроллеры приложения, отвечающие за вход в систему, каталог, заказы, отчёты и вспомогательные интеграции. Контроллеры не реализуют бизнес-логику напрямую, а передают вычисления сервисному слою.

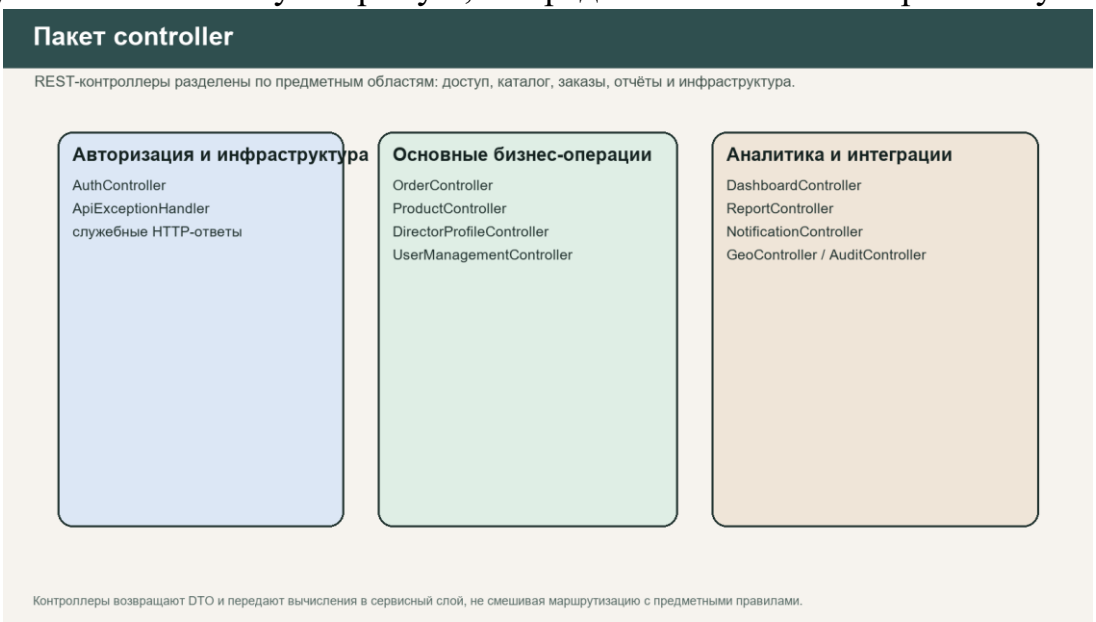

Рисунок 2.6 – Пакет controller

В данный пакет входят:

AuthController — отвечает за авторизацию пользователя, проверку учётных данных и выдачу JWT-токена после успешного входа.

DirectorProfileController — обслуживает профиль директора магазина, адреса доставки и данные, которые отображаются в публичном frontend.

OrderController — основная точка работы с заказами, ручным и автоматическим назначением водителей, preview транспортной задачи и подтверждением маршрутов.

ProductController — сопровождает каталог продукции, фильтрацию и редактирование товарных позиций.

UserManagementController — используется для регистрации директоров магазинов, просмотра списка водителей и сопровождения учётных записей.

DashboardController — возвращает агрегированные показатели по статусам заказов и общую сводку для менеджерского интерфейса.

ReportController — отвечает за формирование отчётов и выгрузку данных по заказам в файл Excel.

NotificationController — публикует поток уведомлений и события для актуализации пользовательских экранов.

GeoController — выполняет геокодирование адресов и подготовку данных для отображения точек на карте.

Production-конфигурация публичной версии оставляет доступными рабочие endpoint'ы авторизации, каталога, заказов, отчётности и маршрутизации.

Контроллеры возвращают клиенту только те данные, которые требуются конкретной роли. На этом уровне выполняются проверка входных параметров,

преобразование объектов в JSON-ответы и выбор корректных HTTP-кодов, а вся предметная логика подтверждения заказа или назначения маршрута передаётся сервисному слою.

Пакеты config и security содержат механизмы безопасности и конфигурации проекта. Здесь описаны правила доступа к эндпоинтам, настройки JWT, CORS и production-профиль публичной версии.

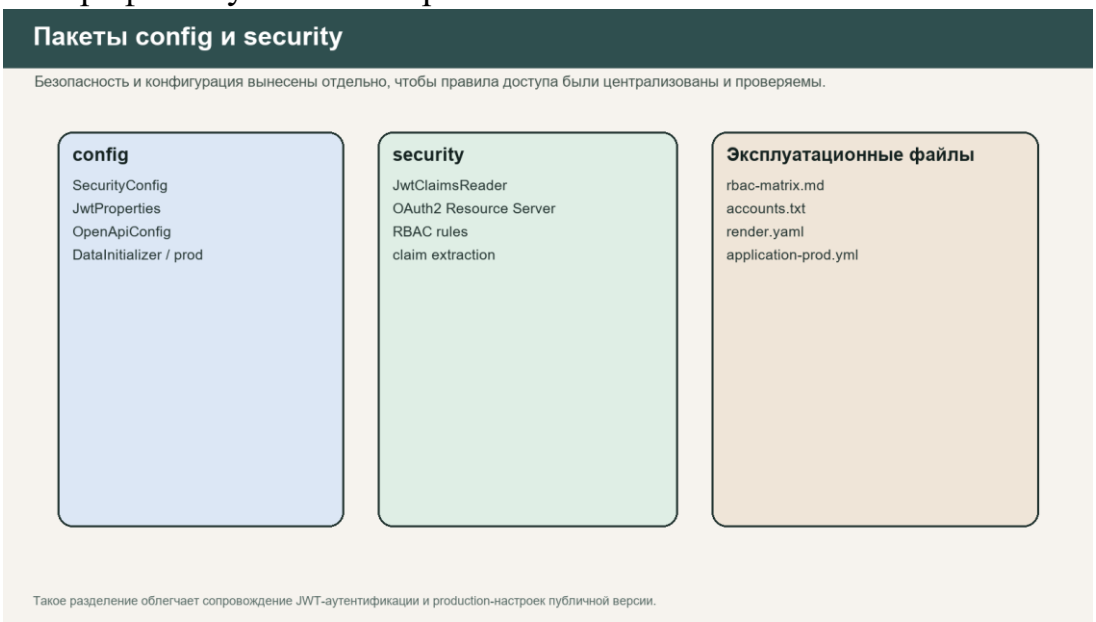

Рисунок 2.7 – Пакеты config и security

За счёт этих пакетов в системе реализуются защита API, привязка ролей к запросам, настройка CORS и профилей окружения, а также выпуск JWT-токенов для клиента.

На уровне безопасности проект проверяет учётные данные пользователя и выпускает JWT-токен для клиента.

Отдельные правила маршрутизации сопоставляют эндпоинты API с разрешёнными ролями пользователей.

Служебные операции блокируются для пользователей с неподходящей ролью, что исключает выполнение чужих сценариев.

Дополнительно конфигурационные пакеты задают production-defaults: служебные сценарии выключены, каталог может наполняться при запуске, а CORS разрешает публичный домен сайта.

Во frontend роль директора реализована в файле DirectorView.jsx. Этот модуль объединяет профиль, адреса, каталог, корзину и историю заказов, а также использует карту для выбора точки доставки.

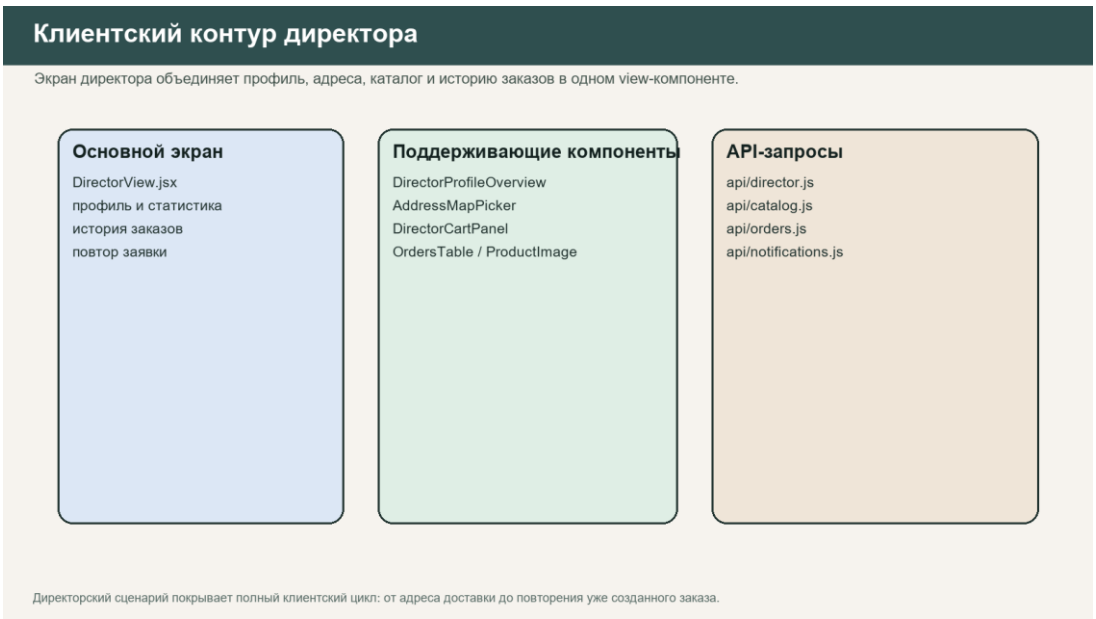

Рисунок 2.8 – Директорский модуль frontend

Пользовательский цикл директора включает проверку профиля, добавление адресов, выбор продукции и формирование заявки. Дополнительно доступны история заказов и сценарий повторного оформления.

В интерфейсе директора выделяется карточка профиля и контактных данных магазина.

Отдельный раздел адресов содержит карту выбора точки доставки и форму сохранения координат.

Каталог обеспечивает фильтрацию продукции, работу с корзиной и расчёт суммы создаваемой заявки.

История заказов отображает статусы документов и позволяет повторить ранее оформленную заявку.

Менеджерский модуль собран вокруг файла ManagerView.jsx. В нём реализованы сводка по системе, kanban-представление статусов заявок, сопровождение каталога, регистрация пользователей и выгрузка Excel-отчётов.

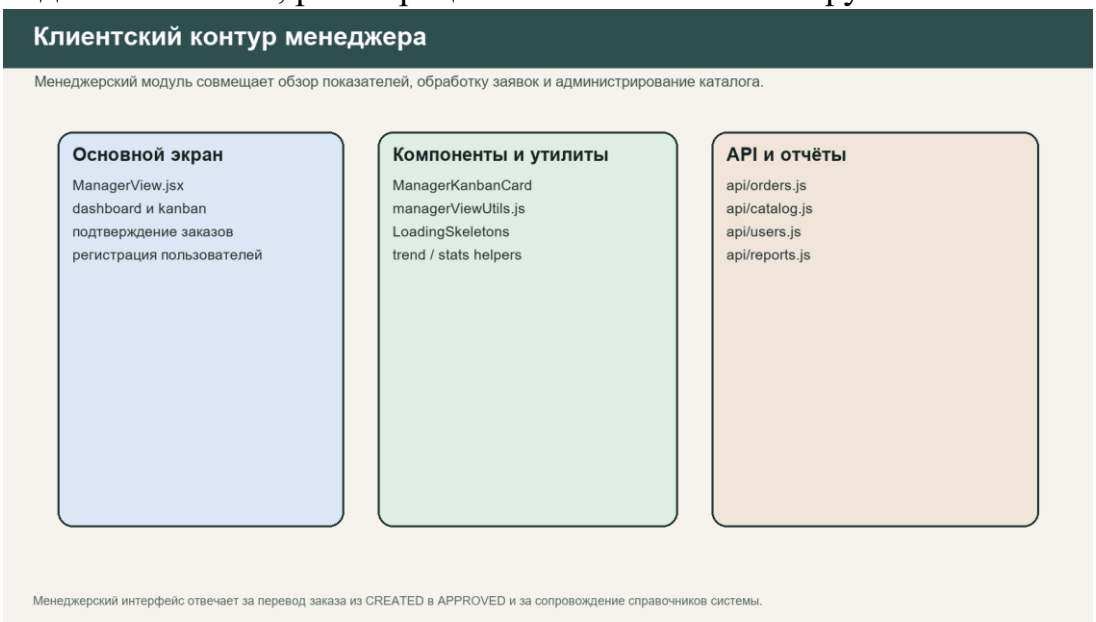

Рисунок 2.9 – Менеджерский модуль frontend

Менеджер отвечает за переход заказа из состояния CREATED в APPROVED, а также за поддержку актуального каталога и учётных записей участников системы.

Dashboard отображает сводную статистику и показатели по заказам.

Панель заявок используется для подтверждения документов и распределения заказов по статусам.

Раздел товаров позволяет добавлять новые позиции, редактировать карточки и контролировать остатки продукции.

Раздел пользователей используется для регистрации новых директоров магазинов и сопровождения учётных записей.

Экран отчётов формирует Excel-выгрузки по выбранному периоду и текущему статусу заказа.

Водительский сценарий сосредоточен в файле DriverView.jsx. На экране водителя отображаются только назначенные ему точки маршрута, последовательность доставок и кнопки завершения заказов.

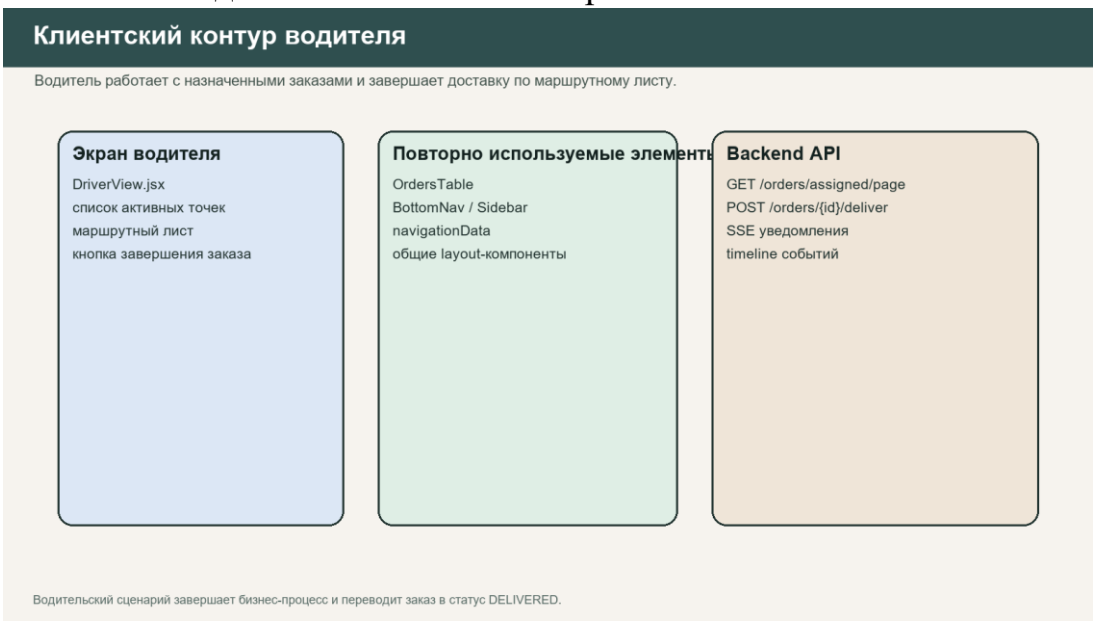

Рисунок 2.10 – Водительский модуль frontend

Этот модуль завершает бизнес-процесс: после подтверждения доставки документ переводится в итоговый статус DELIVERED, а интерфейсы менеджера и директора получают обновлённые данные.

Водитель получает список точек маршрута, адреса доставки, параметры заказов и элементы навигации по закреплённому рейсу.

Тем самым водительский интерфейс выступает не самостоятельным каталогом операций, а рабочим маршрутным листом, который должен быть максимально простым и быстрым в использовании во время фактической доставки.

Пакет логиста реализует один из самых важных участков проекта — построение и подтверждение транспортного плана. Он объединяет таблицу заказов, ручное назначение, выбор водителей и карту маршрутов.

Рисунок 2.11 – Логистический модуль frontend

В логистическом контуре выполняется получение всех заказов со статусом APPROVED.

Пользователь выбирает водителей, которые будут участвовать в расчёте транспортной задачи.

Далее система строит preview транспортной задачи без записи данных в базу.

На карте визуализируются точки доставки и маршруты каждого водителя.

После подтверждения рассчитанного варианта заказы переводятся в состояние ASSIGNED.

Ключевое достоинство логистического модуля заключается в разделении режимов preview и commit: сначала пользователь видит безопасный предварительный расчёт с картой и сводной нагрузкой, и только после подтверждения назначение водителей фиксируется в базе данных.

Общие API, компоненты и эксплуатационные скрипты вынесены в отдельные общие модули. Это позволяет повторно использовать сетевую логику, элементы навигации и deploy-настройки между всеми ролями приложения.

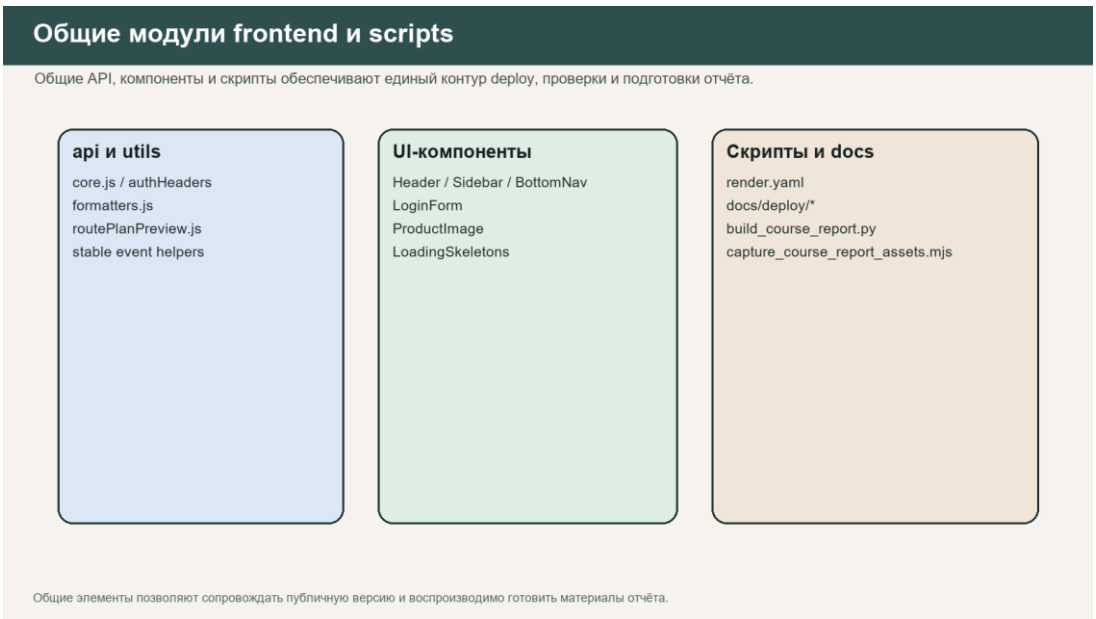

Рисунок 2.12 – Общие модули frontend и scripts

Такое разделение упрощает сопровождение проекта: backend, frontend и сценарии защиты можно развивать независимо, при этом пользовательские потоки остаются согласованными.

К общим модулям относятся клиент API и функции авторизованных HTTP-запросов.

Отдельно вынесены компоненты навигации, карточки статистики и базовые элементы интерфейса.

Deploy-конфигурация, сценарии подготовки отчётных данных и эксплуатационная документация также расположены в общем слое проекта.

Дополнительный слой документации содержит материалы по публичному развёртыванию и проверке задеплоенной версии.

2.3 Результат работы программы

Результат запуска веб-приложения представлен на рисунке 2.13. После старта пользователь попадает на экран авторизации, а затем, в зависимости от своей роли, получает доступ только к разрешённым разделам интерфейса.

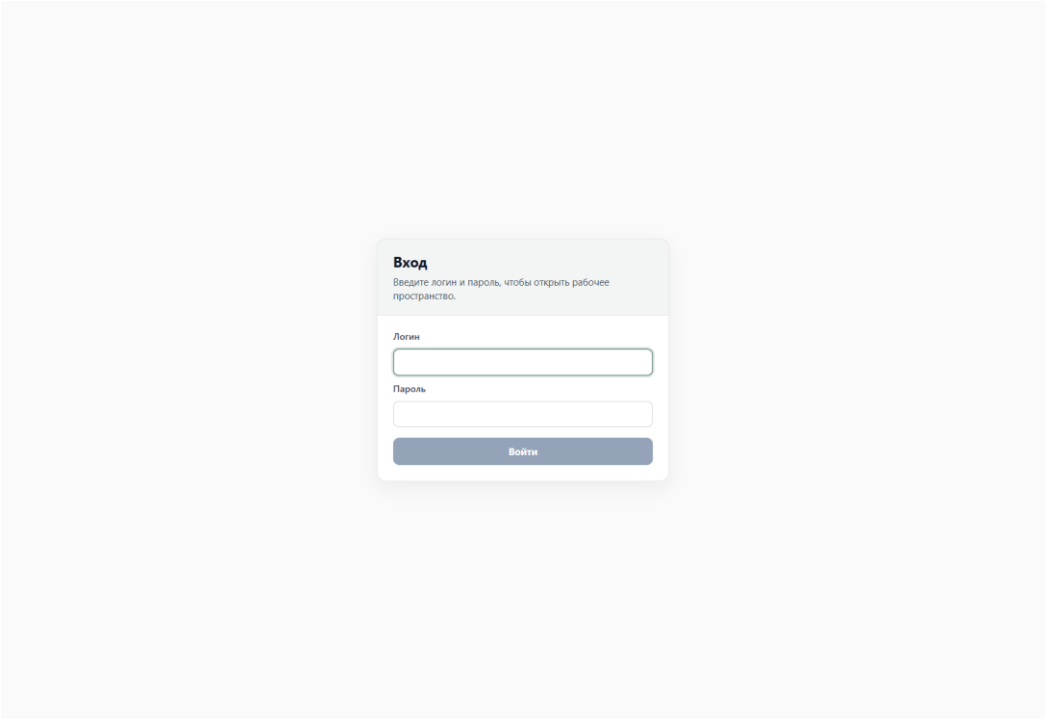

Рисунок 2.13 – Результат запуска программы

Контрольная проверка проекта выполнялась через публичную версию сайта https://farmer.indevs.in. Статическая клиентская часть обращается к backend API https://farm-sales-backend.onrender.com/api, где работают авторизация, каталог, заказы, отчёты и логистические расчёты. В проверочном снимке публичного API система содержит 30 адресов магазинов, каталог из 250 товарных позиций и 32 заказа; из них 29 находятся в статусе ASSIGNED, 3 — DELIVERED, 0 — APPROVED.

Для нового запуска preview на момент проверки нет заказов в статусе APPROVED, поскольку основной логистический набор уже распределён между водителями. Поэтому preview транспортной задачи корректно вернул 0 запланированных документов, 0 нераспределённых документов и суммарный оценочный километраж 0.0 км.

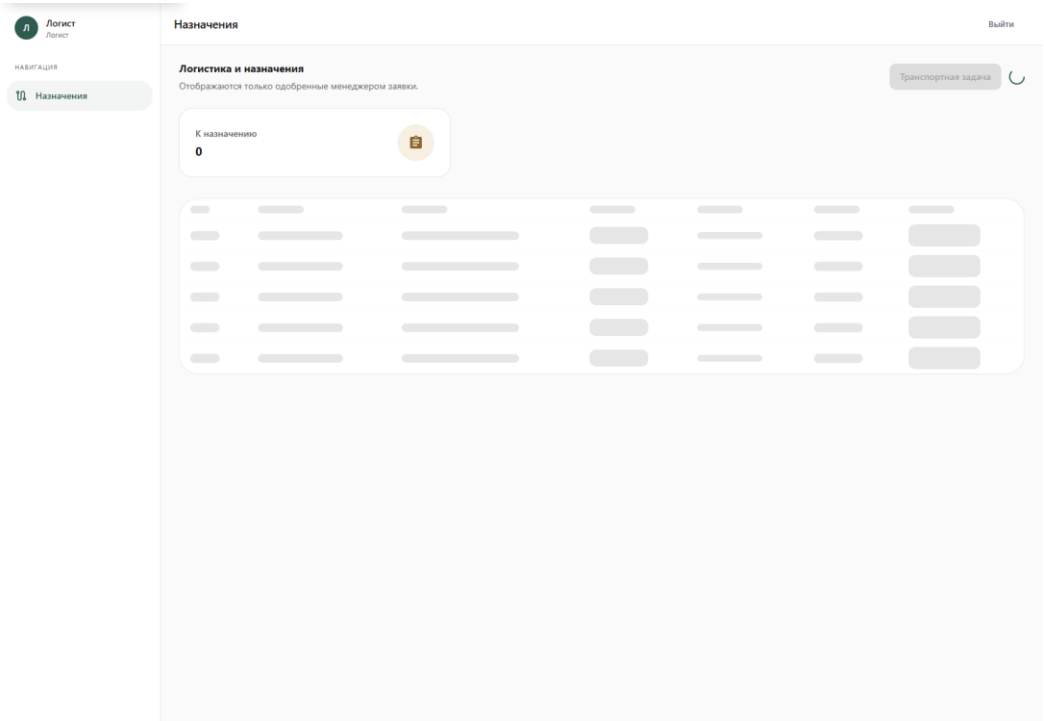

Рисунок 2.14 – Состояние транспортного модуля без новых заказов для preview

Сводка по маршрутам показала текущее состояние публичной базы: новых заказов в статусе APPROVED для повторного расчёта не было; действующий маршрутный контур содержит 29 назначенных заказов и 3 доставленных. При появлении новых заказов в статусе APPROVED этот же экран используется для расчёта маршрутов, а менеджерский раздел отчётности остаётся доступным для выгрузки данных по заказам.

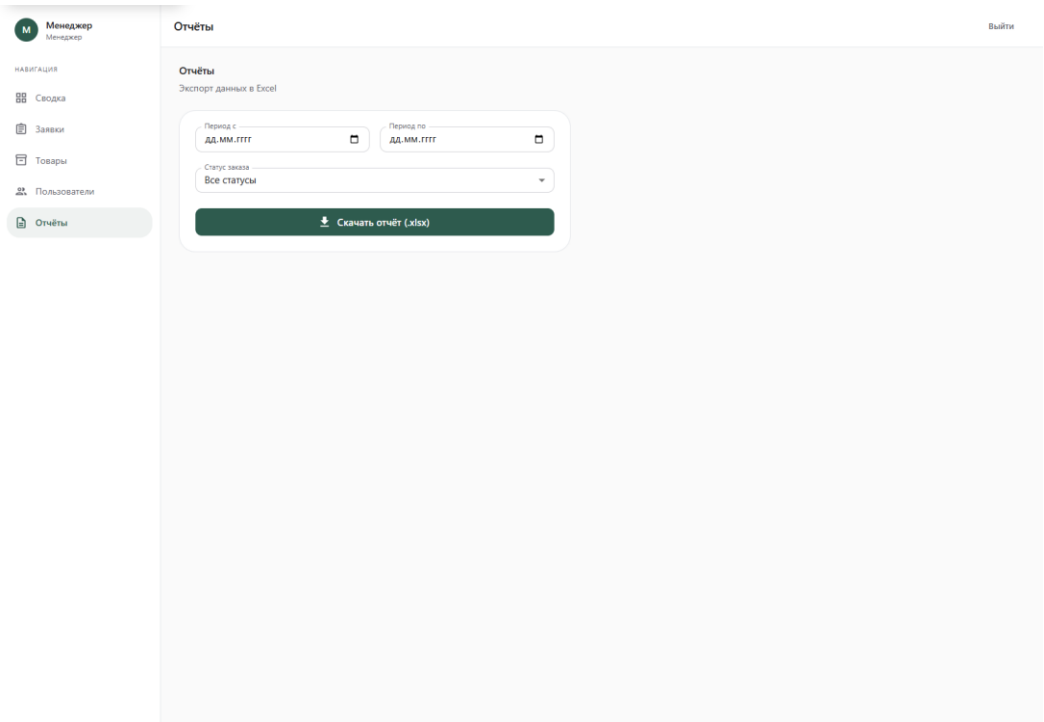

Рисунок 2.15 – Формирование отчётности менеджером

3 Инструкция по пользованию проектом

Для начала работы необходимо открыть публичную версию сайта по адресу https://farmer.indevs.in. Пользователь взаимодействует только с web-интерфейсом, а серверная часть и база данных работают на удалённом backend-контуре (рисунок 3.1).

Рисунок 3.1 – Открытие публичной версии сайта

После открытия публичного сайта автоматически становится доступен экран авторизации, приведённый на рисунке 3.2. Для входа используются роли из файла accounts.txt: директор магазина, менеджер, логист и водитель.

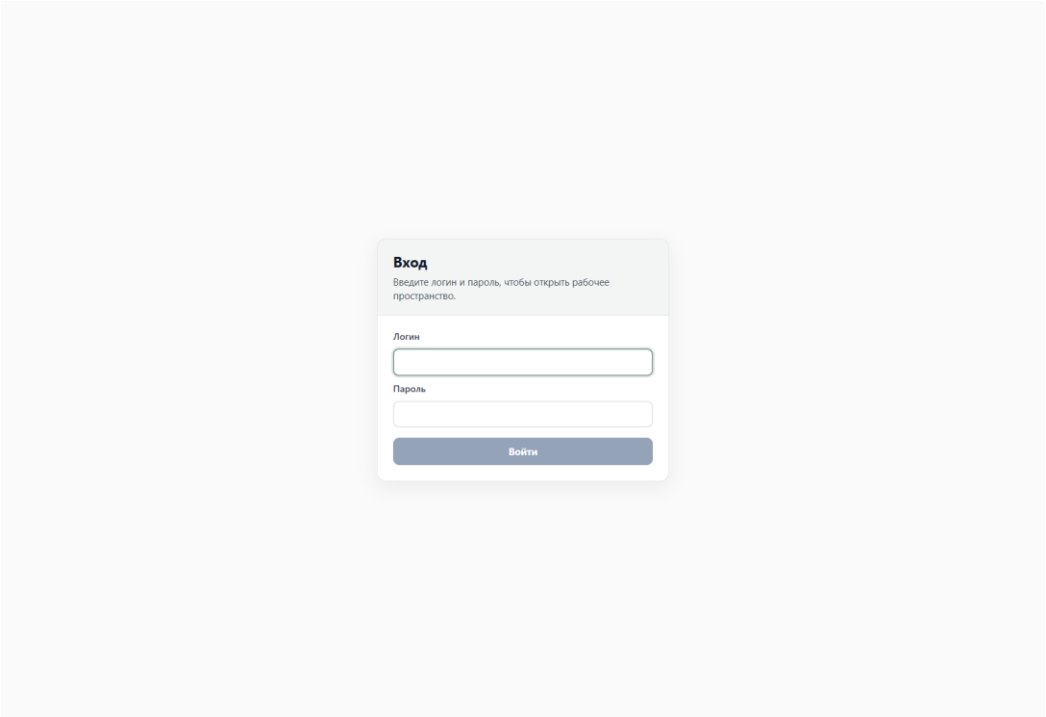

Рисунок 3.2 – Авторизация

После успешного входа директор магазина попадает в свой личный кабинет, где может проверить контактные данные, статистику по заказам и текущую

активность. При необходимости на этом же экране редактируются профильные сведения (рисунок 3.3).

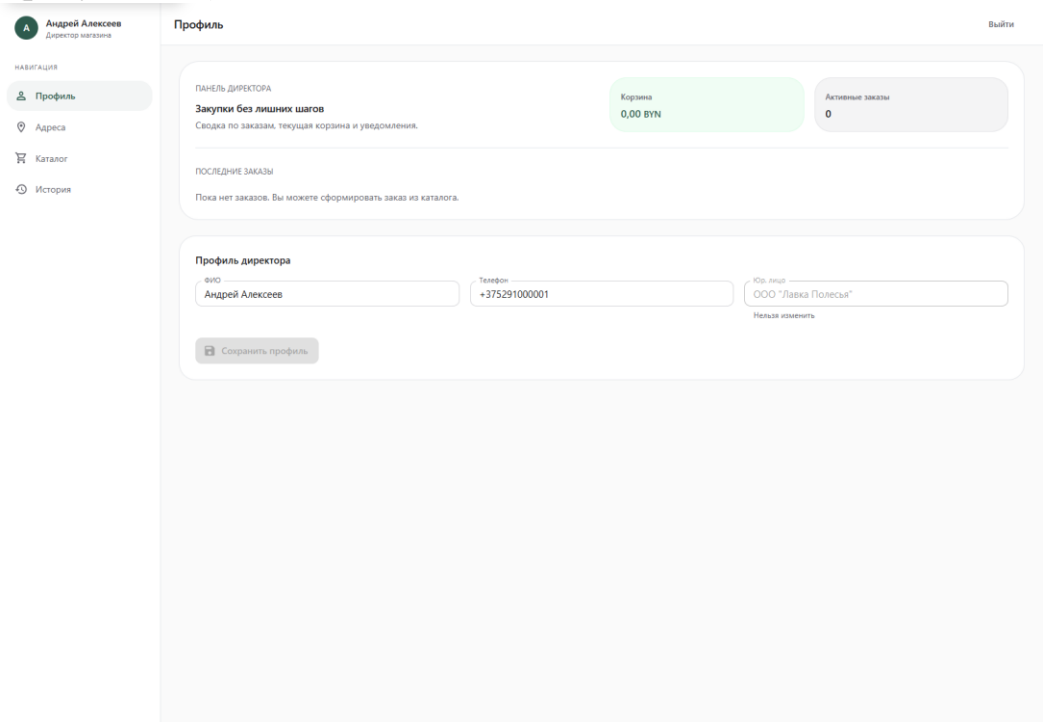

Рисунок 3.3 – Личный кабинет директора магазина

Для подготовки заказа пользователь открывает вкладку адресов доставки, указывает нужную точку на карте и сохраняет её для дальнейшего оформления заявок. Карта позволяет выбрать место доставки и контролировать корректность координат (рисунок 3.4).

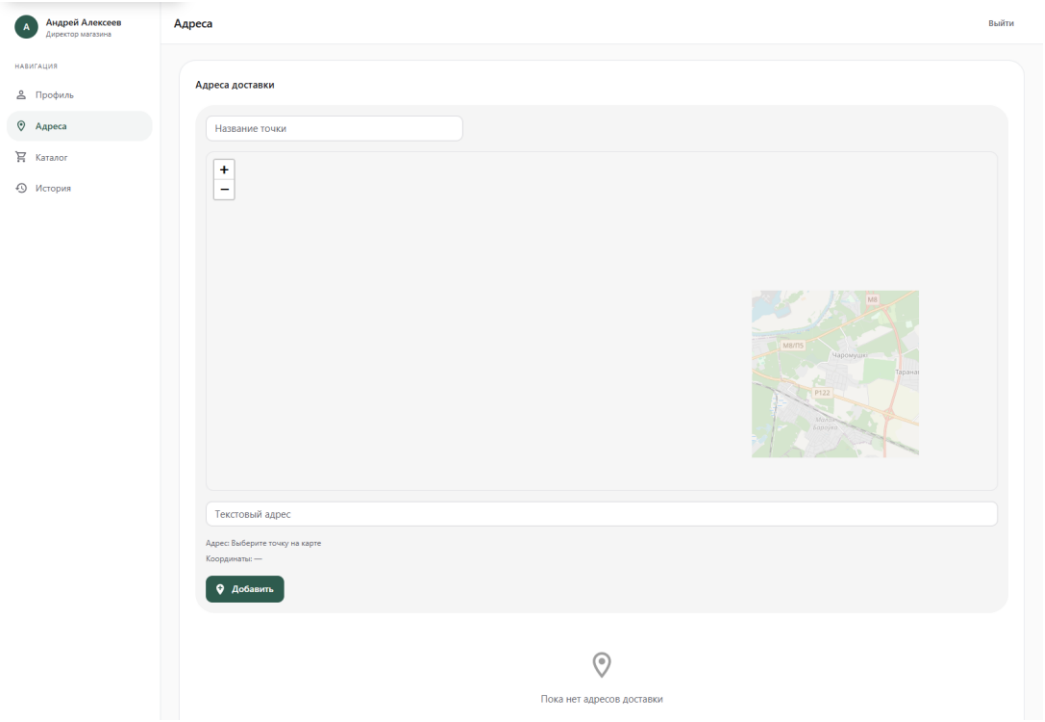

Рисунок 3.4 – Страница выбора адреса доставки

После этого директор переходит в каталог фермерской продукции, выбирает позиции, указывает количество и формирует заявку. Система сразу отображает итоговую сумму заказа и состояние корзины (рисунок 3.5).

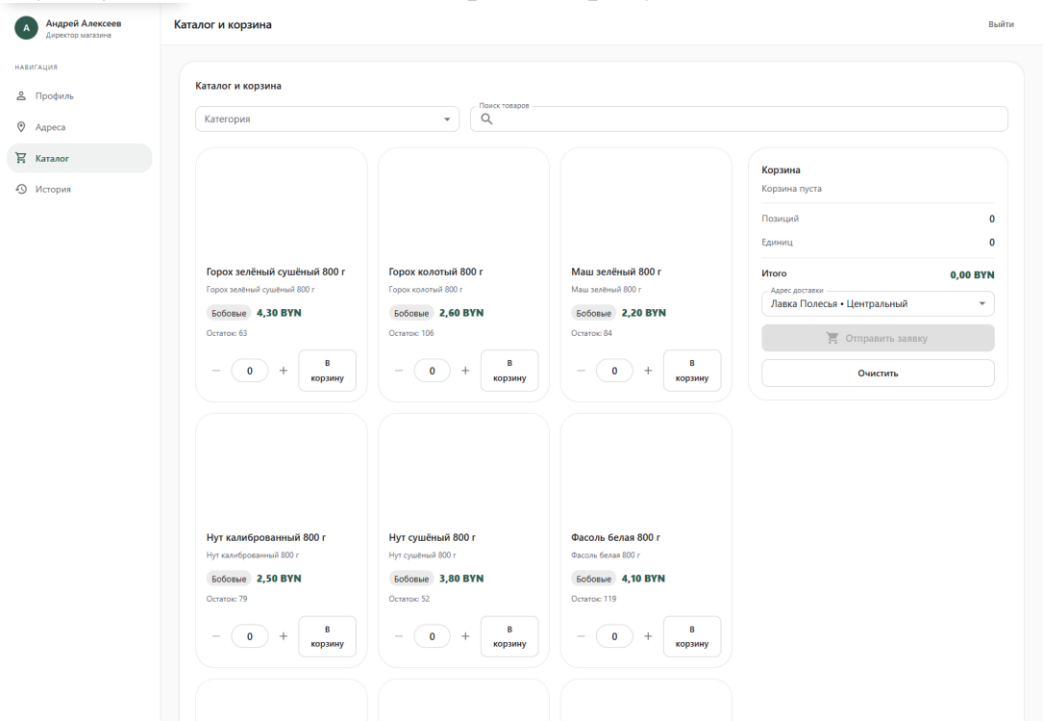

Рисунок 3.5 – Страница каталога продукции

Оформленные документы доступны в истории заказов. Здесь пользователь может отследить смену статуса и, при необходимости, повторить ранее созданную заявку, не заполняя состав заказа заново (рисунок 3.6).

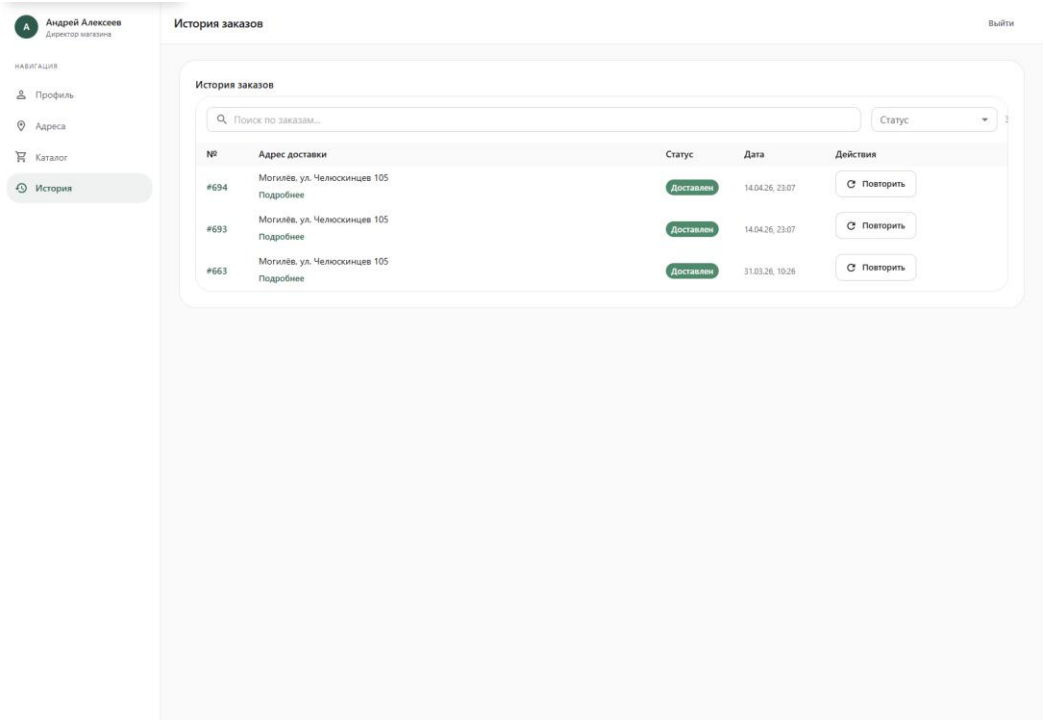

Рисунок 3.6 – История заказов и повторное оформление

Следующим участником процесса является менеджер. После входа под своей учётной записью он получает доступ к обзорной панели системы, где отображаются

показатели по статусам заказов и вспомогательные виджеты аналитики (рисунок 3.7).

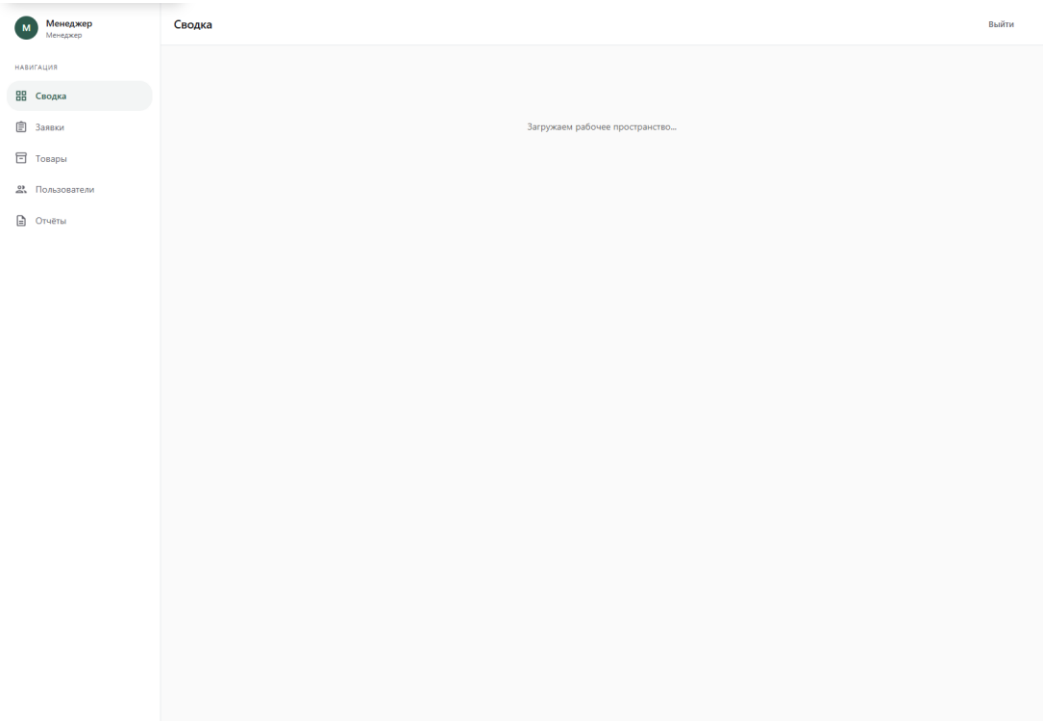

Рисунок 3.7 – Панель управления менеджера

Во вкладке заявок менеджер просматривает поступившие документы и подтверждает те из них, которые готовы к передаче в логистику. Именно на этом этапе заказ переводится из CREATED в APPROVED (рисунок 3.8).

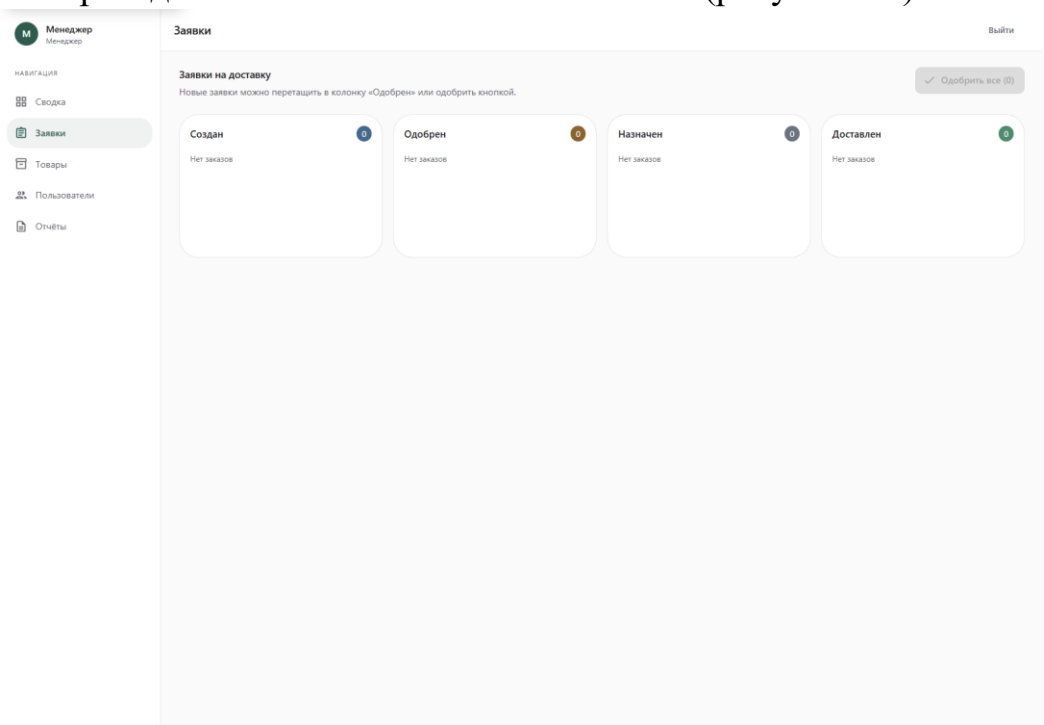

Рисунок 3.8 – Панель управления заказами для подтверждения

Для сопровождения каталога менеджер использует вкладку товаров. Здесь можно просматривать ассортимент, добавлять новые позиции, редактировать карточки и контролировать остатки продукции (рисунок 3.9).

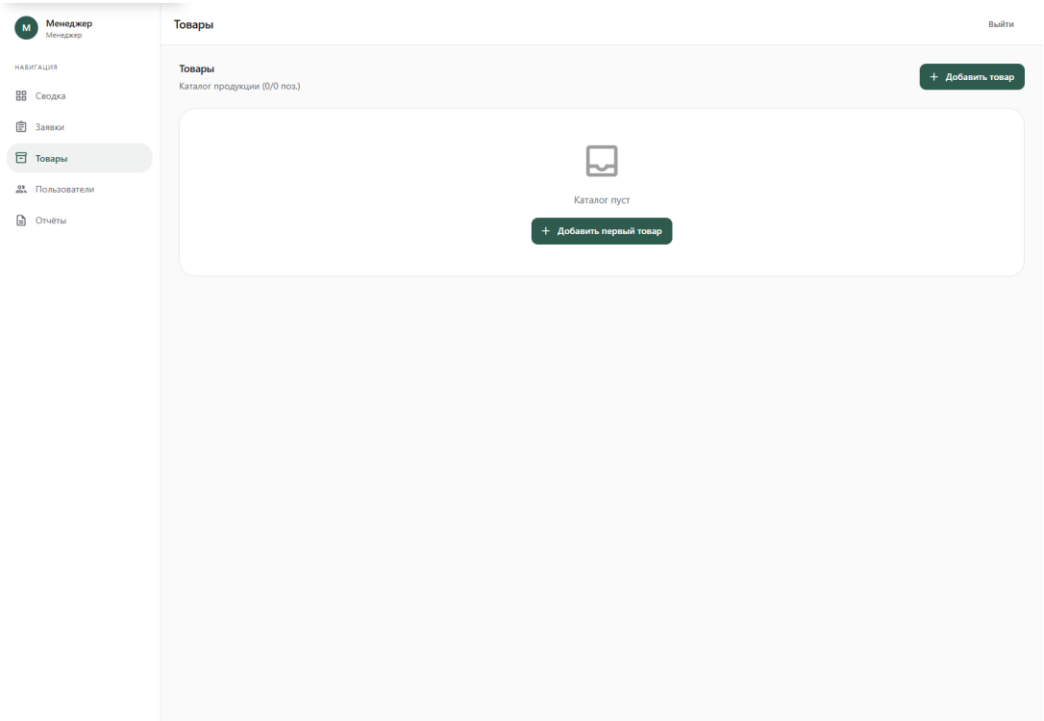

Рисунок 3.9 – Страница управления продуктами

Во вкладке пользователей менеджер регистрирует директоров магазинов и контролирует список действующих учётных записей. Вкладка отчётов позволяет сформировать выгрузку заказов в Excel по выбранному периоду и статусу (рисунок 3.10 и рисунок 3.11).

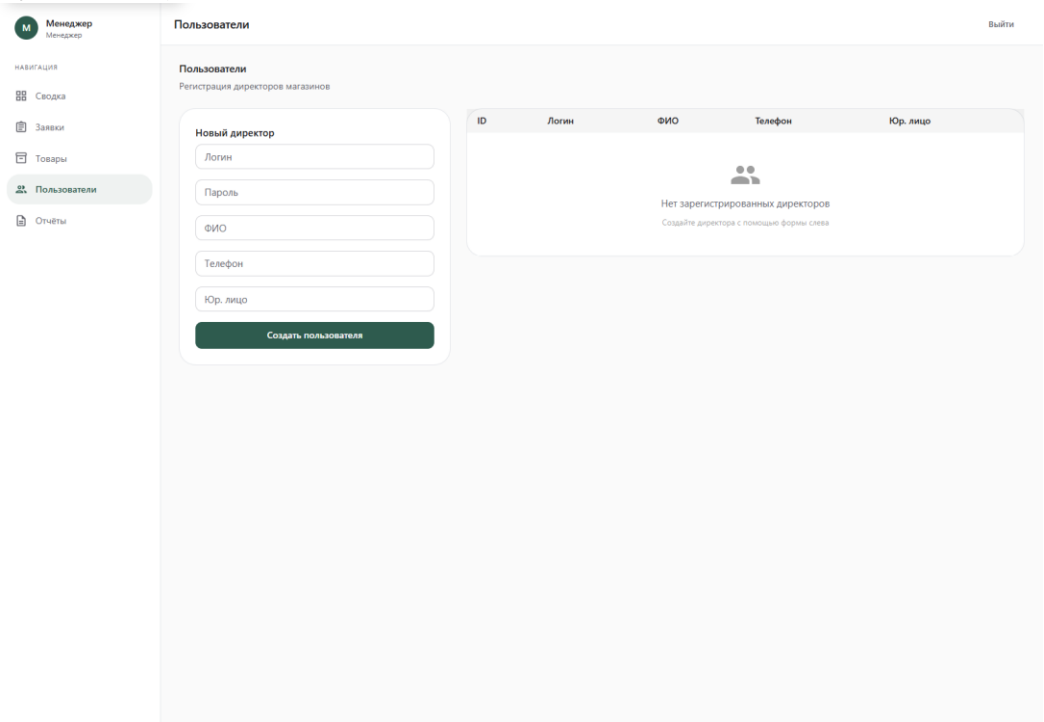

Рисунок 3.10 – Страница управления пользователями

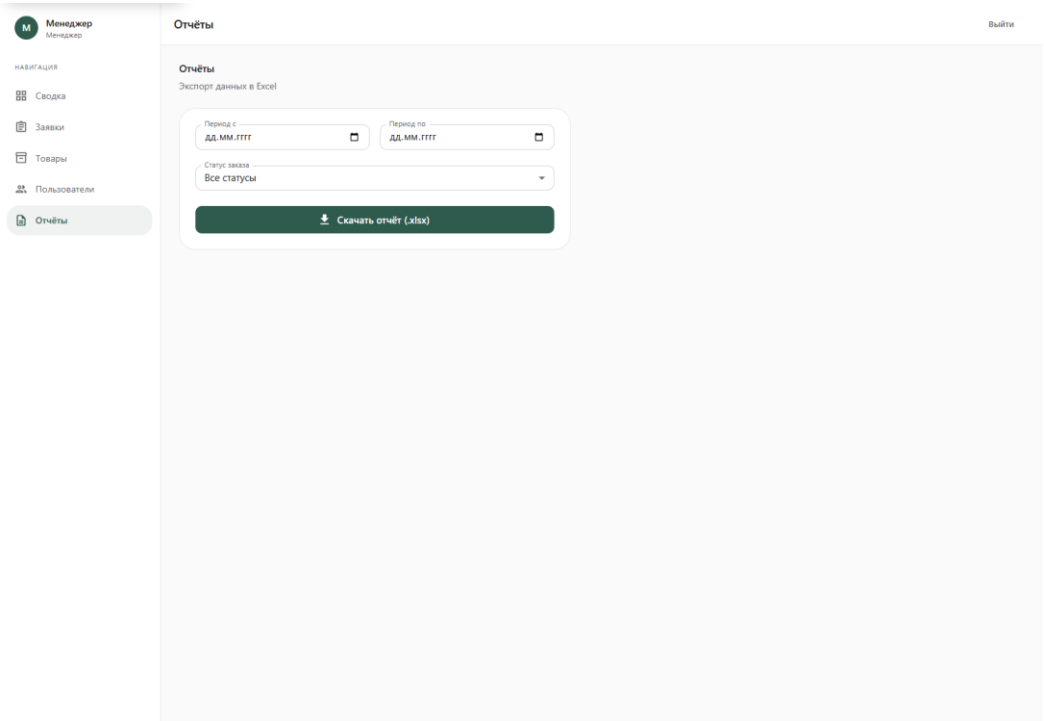

Рисунок 3.11 – Страница формирования отчётов

После подтверждения заказа менеджером дальнейшая обработка переходит к логисту. Логист работает в собственном разделе, где отображаются одобренные и уже назначенные документы, а также состояние маршрутов водителей (рисунок 3.12).

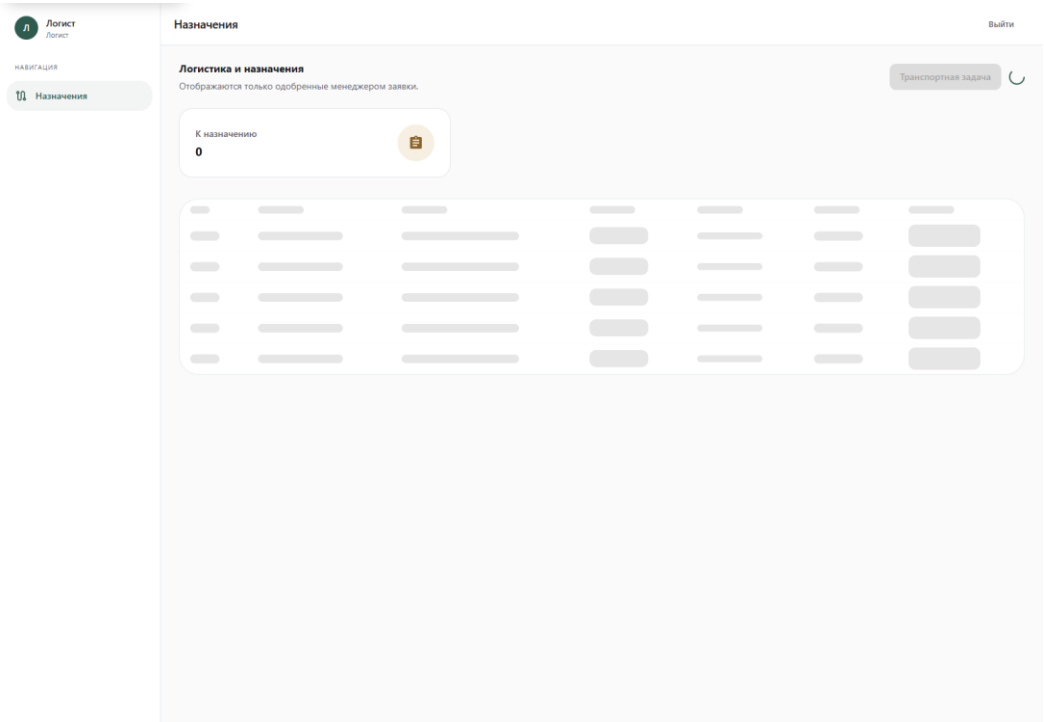

Рисунок 3.12 – Интерфейс логиста

Когда в системе появляются новые заявки в статусе APPROVED, логист запускает транспортную задачу из этого же раздела. В текущем публичном состоянии таких заявок нет, потому что 29 заказов уже назначены водителям, а 3

завершены; экран показывает недоступность повторного расчёта для пустой очереди (рисунок 3.13).

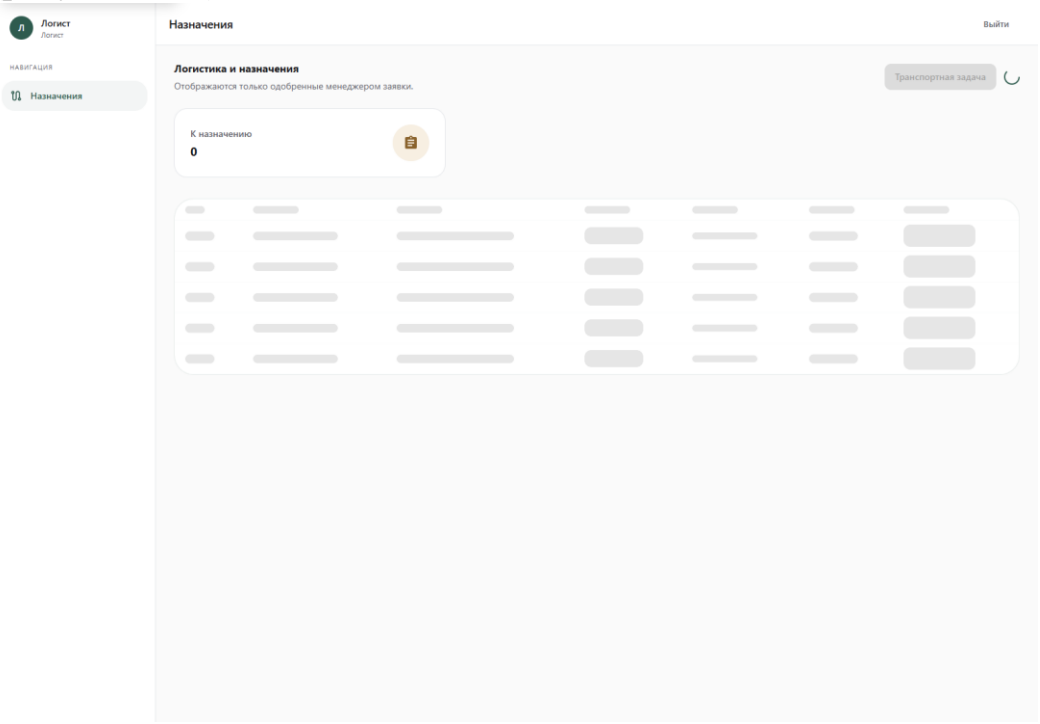

Рисунок 3.13 – Интерфейс логиста без новых заявок для расчёта

На странице предварительного просмотра отображаются выбранные водители, параметры расчёта, количество рейсов и карта с маршрутами. Если в системе есть новые заявки в статусе APPROVED, логист может утвердить распределение и перевести их в статус ASSIGNED; при отсутствии такой очереди экран показывает корректное состояние без нового расчёта (рисунок 3.14).

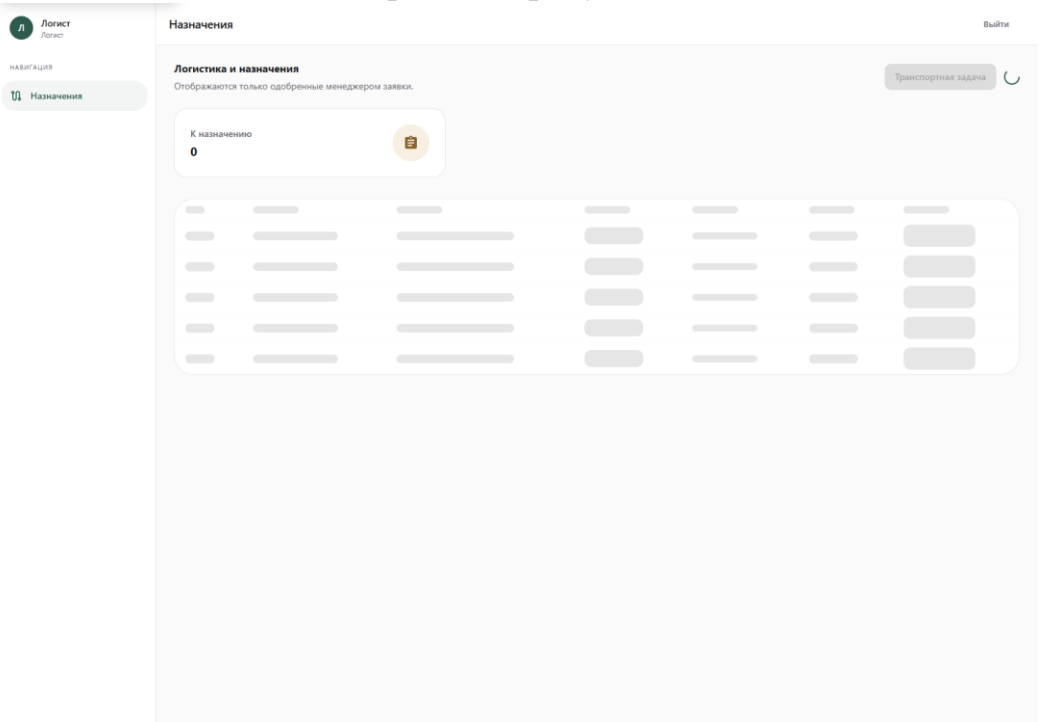

Рисунок 3.14 – Транспортный модуль без новых заказов для preview

После завершения этапа логистического распределения водитель входит в систему под своей учётной записью. В его панели управления отображаются все закреплённые точки маршрута, адреса, параметры заказов и кнопки завершения доставки (рисунок 3.15).

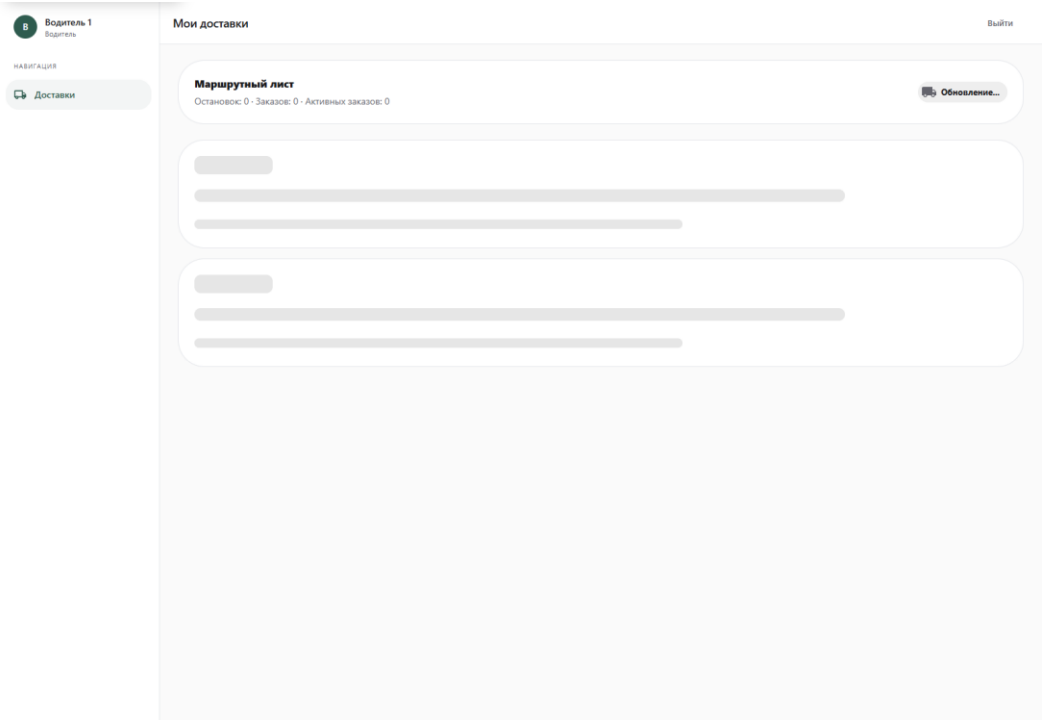

Рисунок 3.15 – Панель управления водителя

После доставки директор магазина может снова открыть историю заказов и убедиться, что документ перешёл в итоговый статус DELIVERED. При необходимости этот же экран используется как точка повторного оформления, что завершает полный пользовательский цикл системы (рисунок 3.16).

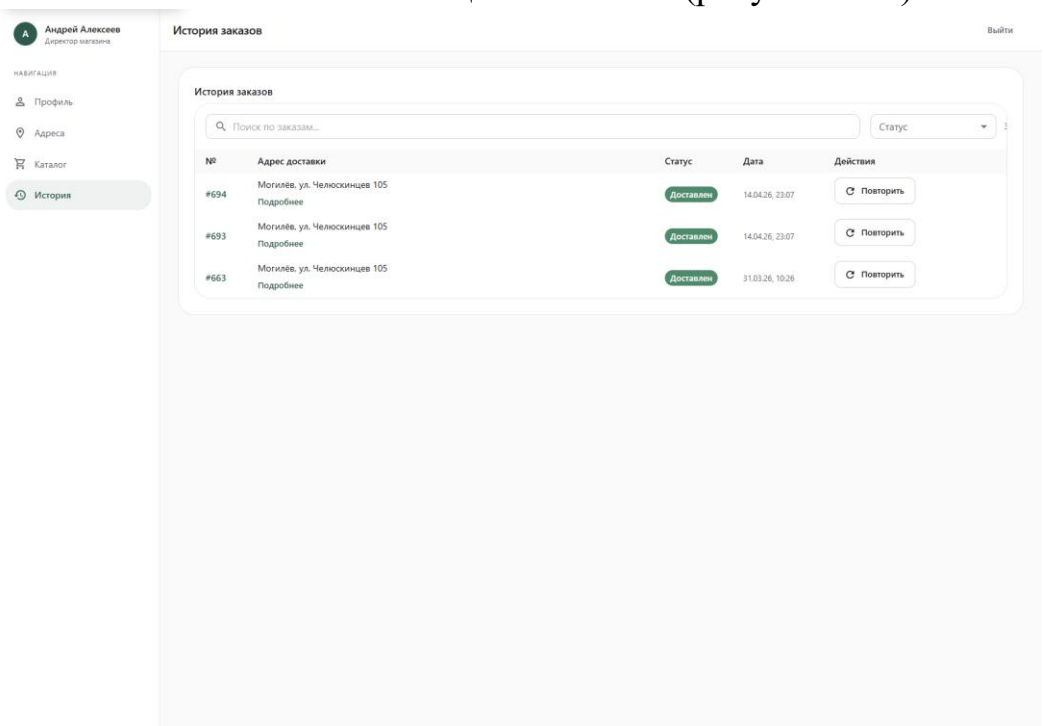

Рисунок 3.16 – Контроль итогового статуса в истории заказов

Заключение

В ходе выполнения курсового проекта было успешно разработано сетевое приложение для автоматизации работы отдела сбыта фермерского хозяйства. Приложение включает ролевую авторизацию, каталог продукции, историю заказов, подтверждение заявок, транспортную задачу и маршрутный лист водителя.

Использование технологий Java 21, Spring Boot, React, Vite и PostgreSQL позволило создать понятный интерфейс и надёжную серверную часть, обеспечивающую высокую производительность и безопасность данных. В проекте реализована модульная структура, облегчающая дальнейшее сопровождение и расширение функциональности.

Разработка базы данных и архитектуры приложения позволила эффективно организовать хранение и обработку информации о пользователях, заказах и товарах. Дополнительным преимуществом проекта стала публичная развёрнутая версия сайта, через которую можно проверить работу системы непосредственно в браузере.

Таким образом, разработанное приложение представляет собой практический инструмент для повышения эффективности работы отдела сбыта и улучшения координации между директором магазина, менеджером, логистом и водителем.

Список использованных источников

1 Документация по Spring Boot [Электронный ресурс]. – Режим доступа: https://docs.spring.io/spring-boot/docs/current/reference/html/. – Дата доступа: 23.04.2026.

2 Документация по React [Электронный ресурс]. – Режим доступа: https://react.dev/. – Дата доступа: 23.04.2026.

3 Документация по Vite [Электронный ресурс]. – Режим доступа: https://vite.dev/guide/. – Дата доступа: 23.04.2026.

4 Документация по PostgreSQL [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs/. – Дата доступа: 23.04.2026.

5 Документация по Flyway [Электронный ресурс]. – Режим доступа: https://documentation.red-gate.com/fd. – Дата доступа: 23.04.2026.

6 Документация по Material UI [Электронный ресурс]. – Режим доступа: https://mui.com/. – Дата доступа: 23.04.2026.

7 Документация по Leaflet [Электронный ресурс]. – Режим доступа: https://leafletjs.com/reference.html. – Дата доступа: 23.04.2026.

8 Документация по Spring Security [Электронный ресурс]. – Режим доступа: https://docs.spring.io/spring-security/reference/. – Дата доступа: 23.04.2026.

9 Материалы проекта Farm Sales: README.md, docs/PROJECT_OVERVIEW.md, docs/defense/scenario.md, docs/defense/transport-task.md.

10 СТП СМК 7.5.3-02-2023 Общие требования и правила оформления учебных текстовых документов [Электронный ресурс]. – Режим доступа: https://www.bgut.by/sites/default/files/userfiles/rio/files/stp_smk.pdf. – Дата доступа: 23.04.2026.